SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

## Department of Computer Science and Engineering

| | |
|---|---|
| **Course Code & Name** | CSA06 – Design and Analysis of Algorithms |
| **Assessment** | CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform |
| **Course Outcome Assessed** | Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. |
| **Duration** | 3 Days |
| **Mode** | Individual |
| **Student Reg. No.** | 192473019 |
| **Student Name** | Y Sai Chetan |

**Instructions:** Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute $10^8$ basic operations per second.

### IT APPLICATION-BASED PROBLEM

**Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform:** ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

**Module M1 – Duplicate-Listing Detector:** Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

**Module M2 – API Rate Limiter:** The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j]) END FOR                          // basic operation
   i ← i × 2 END WHILE
```

**Module M3 – Product Search:** Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by

comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

**Module M4 – Recommendation Engine:** The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

**Module M5 – Sales Analytics Engine:** The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

**(a) Understand the Problem:** For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

**(b) Analyse the Iterative Modules:** For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

**(c) Solve a Recurrence by Substitution:** Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

**(d) Apply the Master Theorem:** For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

**(e) Analyse Best, Average and Worst Cases:** For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

**(f) Prove an Asymptotic Bound:** Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants $c_1$, $c_2$ and $n_0$, and show that f(n) is not O(n).

**(g) Scalability Decision:** Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

## Rubrics – Application-Based Question

| Criteria | Weightage (%) | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20 | Clearly understands the problem and accurately identifies all requirements | Good understanding with minor gaps | Basic understanding; some requirements unclear | Poor understanding or misinterpretation of the problem |
| Application of Concepts | 30 | Accurately applies appropriate concepts to solve complex problems | Applies relevant concepts correctly with minor errors | Applies basic concepts with limited accuracy | Fails to apply appropriate concepts |
| Problem-Solving Approach | 20 | Logical, well-structured, and efficient approach | Mostly logical approach with minor gaps | Basic approach with some inconsistencies | Illogical or unclear approach |

| Criteria | Weightage (%) | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Accuracy of Solution | 20 | Completely correct solution with proper justification | Mostly correct with minor errors | Partially correct solution | Incorrect or incomplete solution |
| Clarity | 10 | Clear, well-organized, and properly explained solution | Understandable with minor clarity issues | Limited clarity and explanation | Poorly presented and difficult to understand |

## Mark Evaluation:

| Criteria | Weightage (%) | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| Understanding of Problem Context | 20 | | | | |
| Application of Concepts | 30 | | | | |
| Problem-Solving Approach | 20 | | | | |
| Accuracy of Solution | 20 | | | | |
| Clarity | 10 | | | | |
| **Total Marks** | | | | | |

### Code of Conduct

I Y sai chetan (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Y sai chetan  Date: 11-10-2026

# SCALABILITY AUDIT OF "SHOPSPHERE"

## (a) Understanding the Problem

## Module M1: Duplicate-Listing Detector

- Input size: n (number of products).
- Basic operation: the comparison Product[i].title = Product[j].title AND seller = seller.
- Where it occurs: in the IF statement inside the inner loop. It executes once for every pair (i, j) with i < j.
- Expected growth: every product is compared with every later product, so the count grows like $n^2$. If n becomes 10 times bigger, the work becomes about 100 times bigger.

### Module M2: API Rate Limiter

- Input size: n (the maximum window size).
- Basic operation: Audit(Request[j]).
- Where it occurs: inside the inner FOR loop. The inner loop runs i times, and i doubles each round (1, 2, 4, 8, …).
- Expected growth: the rounds add up to about n in total, so growth is linear, not n log n.

### Module M3: Product Search

- Input size: n (number of products or index entries).
- Basic operation: comparing the search key with one entry.
- S1 (sequential scan): one comparison per scanned item, so growth is about n.
- S2 (binary search): one comparison per halving, so growth is about log n.

### Module M4: Recommendation Engine

- Input size: n (customer–product data).
- Basic operation: the merge step, which does n units of work in each recursive call.
- Current design, $T(n) = 4T(n/2) + n$: 4 sub-problems of half size make the number of calls grow fast, so growth is about $n^2$.
- Redesign, $T(n) = 2T(n/2) + n$: only 2 sub-problems, so growth is about n log n.

### Module M5: Sales Analytics Engine

- Input size: n (sales records).
- Basic operation: a comparison made while sorting the records at each level (n log n work per level).
- Expected growth: there are about log n levels and each level costs about n log n, so growth is about $n \log^2 n$.

**Report generator** $f(n) = 3n^2 + 10n \log n + 50$

- Input size: n.
- Basic operation: each counted operation of the routine.
- Expected growth: the $3n^2$ term dominates for large n, so growth is quadratic.

## (b) Analysis of the Iterative Modules

## M1: exact number of comparisons

- When i = 0, j runs from 1 to n−1, giving (n−1) comparisons.
- When i = 1, there are (n−2) comparisons.
- This continues until i = n−2, which gives 1 comparison.

Total = (n−1) + (n−2) + … + 2 + 1
= n(n−1)/2
= $(n^2 - n)/2$

- Time complexity: $\Theta(n^2)$. The same count happens in the best, average and worst case, because there is no early exit.
- Space complexity: $\Theta(1)$. Only the variables i and j are used.
- Meaning for the platform: with n = 100,000 there are about $5 \times 10$ comparisons (about 50 s). With n = 1,000,000 there are about $5 \times 10^{11}$ comparisons. Data grows 10×, but time grows 100×.

### M2: exact number of Audit operations

- i takes the values 1, 2, 4, 8, … , 2^(m−1) while i < n.
- In each round the inner loop runs i times, so that round does i Audit operations.
- Number of rounds: m = ⌈log

n⌉. Total = 1 + 2 + 4 + … + 2^(m−1)
= 2^m − 1 (geometric series)

- For n a power of 2: 2^m = n, so the total = n − 1.
- For any n: n ≤ 2^m < 2n, so n − 1 ≤ total ≤ 2n − 2.
- So the total is Θ(n).

Is the team member's claim Θ(n log n) correct? No.
The claim assumes the inner loop runs n times in every round. It actually runs 1, 2, 4, 8, … times. Only the last round is close to n. The earlier rounds together add up to about the same as the last round. This is a geometric series, so the sum is about 2n, not n log n.
The tight bound is Θ(n). (n log n is a valid upper bound but not a tight one.)
Verification for n = 1,024:

- i = 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (10 rounds). When i = 1024 the loop stops.
- Audits = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 128 + 256 + 512 = 1,023 = n − 1
- The team member's formula gives n log n = 1024 × 10 = 10,240, which is about 10 times too large.
- So my result is correct and the claim is wrong.

### (c) Recurrence by Substitution: Search Version S2

T(n) = T(n/2) + 1, with T(1) = 1

**Backward substitution:**
Step 1: T(n) = T(n/2) + 1
Step 2: replace T(n/2) = T(n/4) + 1
T(n) = [T(n/4) + 1] + 1 = T(n/4) + 2
Step 3: replace T(n/4) = T(n/8) + 1
T(n) = [T(n/8) + 1] + 2 = T(n/8) + 3
Pattern after k steps: **T(n) = T(n/2^k) + k**
Stop when n/2^k = 1, which gives 2^k = n, so k = log n.
T(n) = T(1) + log n = 1 + log n
**Exact closed form: T(n) = log n + 1** (for n a power of 2)

| Implementation | Time complexity | Space complexity |
|---|---|---|
| Iterative | Θ(log n) | Θ(1) (only low, high, mid are stored) |
| Recursive | Θ(log n) | Θ(log n) (the call stack is log n levels deep) |

Meaning: for n = 1,000,000, S2 needs only about 20 comparisons.

### (d) Master Theorem

For T(n) = aT(n/b) + f(n), compare f(n) with n^(log_b a):

- Case 1: f(n) is polynomially smaller, so T(n) = Θ(n^(log_b a)).
- Case 2: f(n) = Θ(n^(log_b a)), so T(n) = Θ(n^(log_b a) · log n).
- Case 3: f(n) is polynomially larger (and the regularity condition holds), so T(n) = Θ(f(n)).

## M4: current design T(n) = 4T(n/2) + n

- a = 4, b = 2, f(n) = n
- n^(log_b a) = n^(log 4) = n²
- f(n) = n = O(n^(2−ε)) with ε = 1, so f(n) is polynomially smaller than n². **Case 1.**
- **T(n) = Θ(n²)**
- When the data doubles: T(2n) ≈ (2n)² = 4n², so the time becomes **4 times** larger.

## M4: proposed redesign T(n) = 2T(n/2) + n

- a = 2, b = 2, f(n) = n
- n^(log_b a) = n^(log 2) = n

- f(n) = Θ(n), which is equal to n^(log_b a). **Case 2.**
- **T(n) = Θ(n log n)**

- When the data doubles: $T(2n) = 2n \cdot \log(2n) = 2n \cdot \log n + 2n$, which is only slightly more than **2 times** the old time.

Conclusion: the redesign is much better. Doubling the data costs 4× in the current design but only a little over 2× in the redesign.

## M5: T(n) = 2T(n/2) + n log n

- a = 2, b = 2, f(n) = n log n, n^(log_b a) = n
- Case 1 does not apply, because f(n) is larger than n.
- Case 2 does not apply, because f(n) = n log n is not Θ(n).
- Case 3 does not apply, because f(n)/n = log n is not larger than n^ε for any ε > 0. It is not polynomially larger.
- So **none of the three standard cases applies.**

**Extended Master Theorem:** if f(n) = Θ(n^(log_b a) · log^k n) with k ≥ 0, then T(n) = Θ(n^(log_b a) · log^(k+1) n).

Here k = 1, so **T(n) = Θ(n log² n)**.

**Check using a recursion tree:**

- At level i there are 2^i sub-problems, each of size n/2^i.
- Cost of one sub-problem = (n/2^i) · log(n/2^i), so the cost of level i = n · (log n − i).
- Total = n[log n + (log n − 1) + … + 2 + 1] = n · log n (log n + 1)/2
- This is **Θ(n log² n)**, which matches the extended theorem.

## (e) Best, Average and Worst Cases

**S1: sequential search**

| Case | Input condition | Comparisons |
|---|---|---|
| Best | Product is at the first position | 1 |
| Average (if found) | Product equally likely at any position | (n + 1)/2 |
| Worst | Product is at the last position or not present | n |

**Average case with probability p that the product exists:**

- Found (probability p): the product is equally likely at positions 1 to n, so the average cost is (1 + 2 + … + n)/n = (n+1)/2.
- Not found (probability 1 − p): all n items are checked, so the cost is n.

$A(n) = p \cdot (n + 1)/2 + (1 - p) \cdot n$

For n = 1,000,000 and p = 0.5:

$A = 0.5 \times (1{,}000{,}001/2) + 0.5 \times 1{,}000{,}000$

$= 250{,}000.25 + 500{,}000$

= **750,000.25, which is about 750,000 comparisons**

**S2: binary search**

| Case | Input condition | Comparisons |
|---|---|---|
| Best | The key is the middle element of the index | 1 |
| Average | The key is at a random position | about log n − 1, which is Θ(log n) |
| Worst | The key is found at the last halving, or is absent | ⌊log n⌋ + 1 (about 20 for n = 10 ) |

## (f) Proof of an Asymptotic Bound

Given: $f(n) = 3n^2 + 10n \log n + 50$

**Definition:** $f(n) = \Theta(n^2)$ if there exist constants $c_1, c_2 > 0$ and $n_0$ such that $c_1 n^2 \le f(n) \le c_2 n^2$ for all $n \ge n_0$.

**Lower bound:** all terms are positive, so $f(n) \ge 3n^2$. Take **$c_1 = 3$**.

**Upper bound:** for $n \ge 1$, we have $\log n \le n$ and $1 \le n^2$.

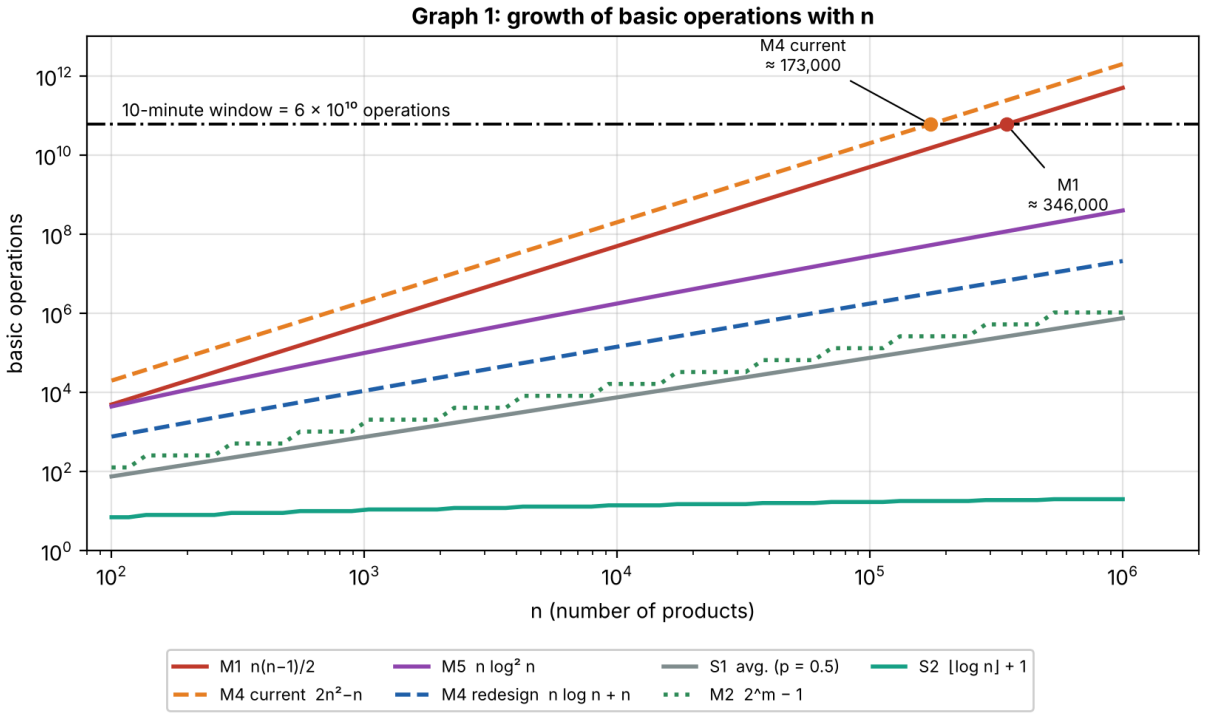
- $10n \log n \le 10n \cdot n = 10n^2$
- $50 \le 50n^2$

So $f(n) \le 3n^2 + 10n^2 + 50n^2 = 63n^2$. Take **$c_2 = 63$**.

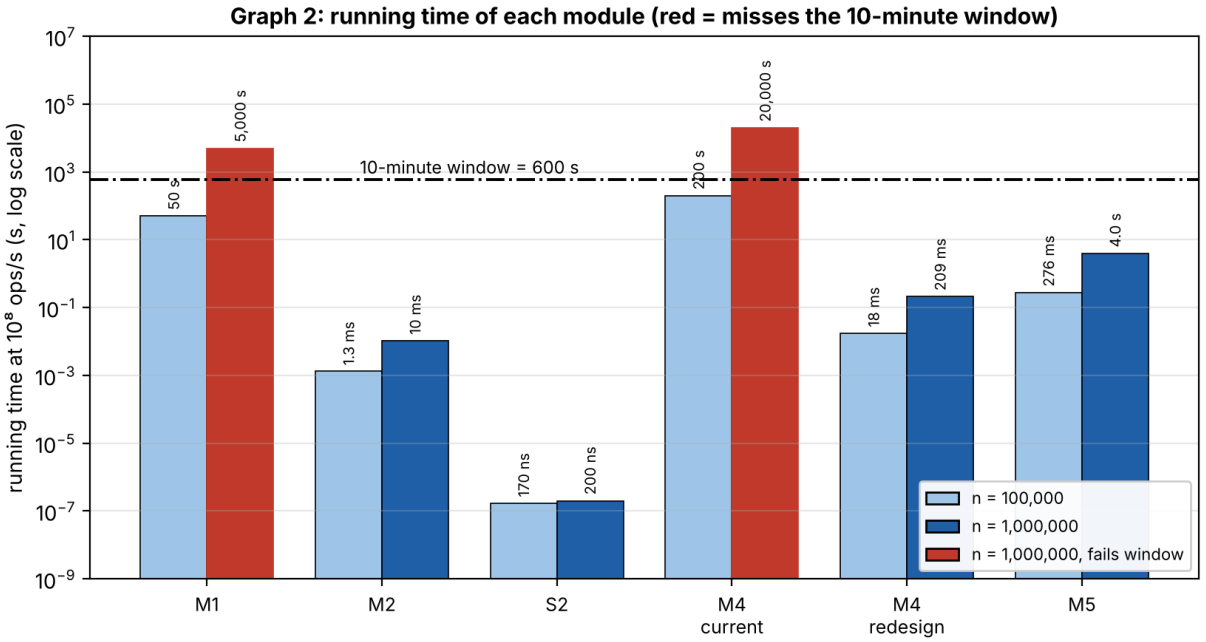
**Constants: $c_1 = 3$, $c_2 = 63$, $n_0 = 1$.**

So $3n^2 \le f(n) \le 63n^2$ for all $n \ge 1$. Therefore **$f(n) = \Theta(n^2)$**. ∎

# GRAPHICAL ANALYSIS OF THE SHOPSPHERE MODULES

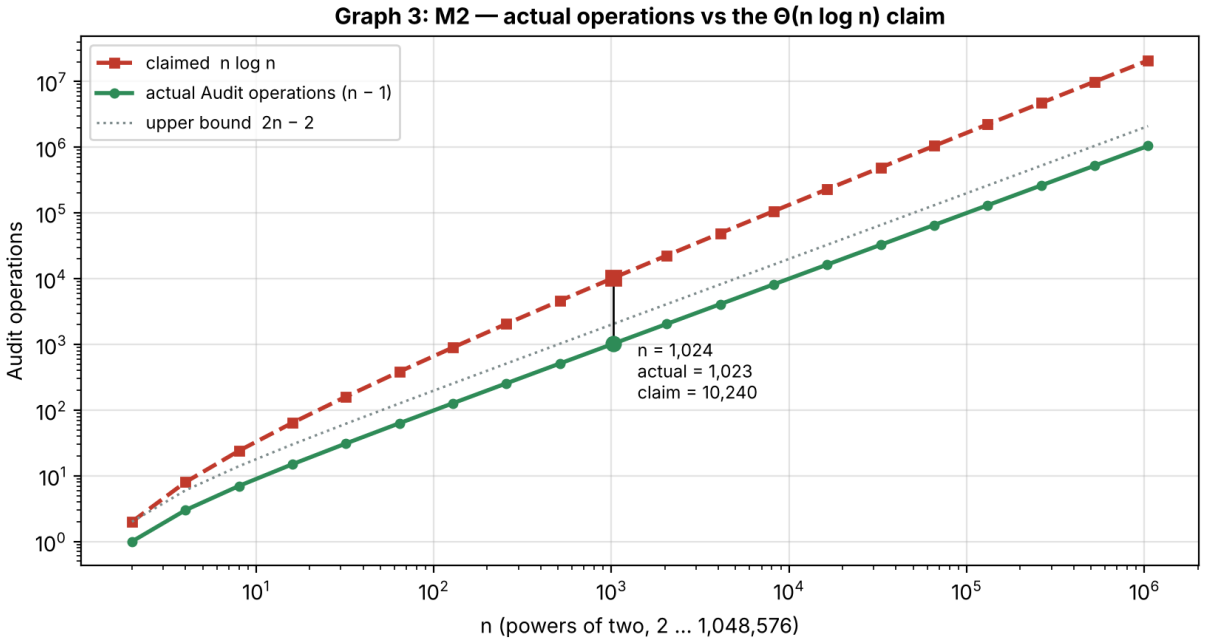



**Reading Graph 1:** the two quadratic modules, M1 and the current M4, are the only curves that reach the 10-minute budget of $6 \times 10^{10}$ operations. M4 (current) crosses it at about 173,000 products and M1 at about 346,000, so M4 (current) already uses a third of the budget at today's 100,000 products, and both fail well before the expected 1,000,000. M5, the M4 redesign, M2, S1 and S2 all stay far below the line.

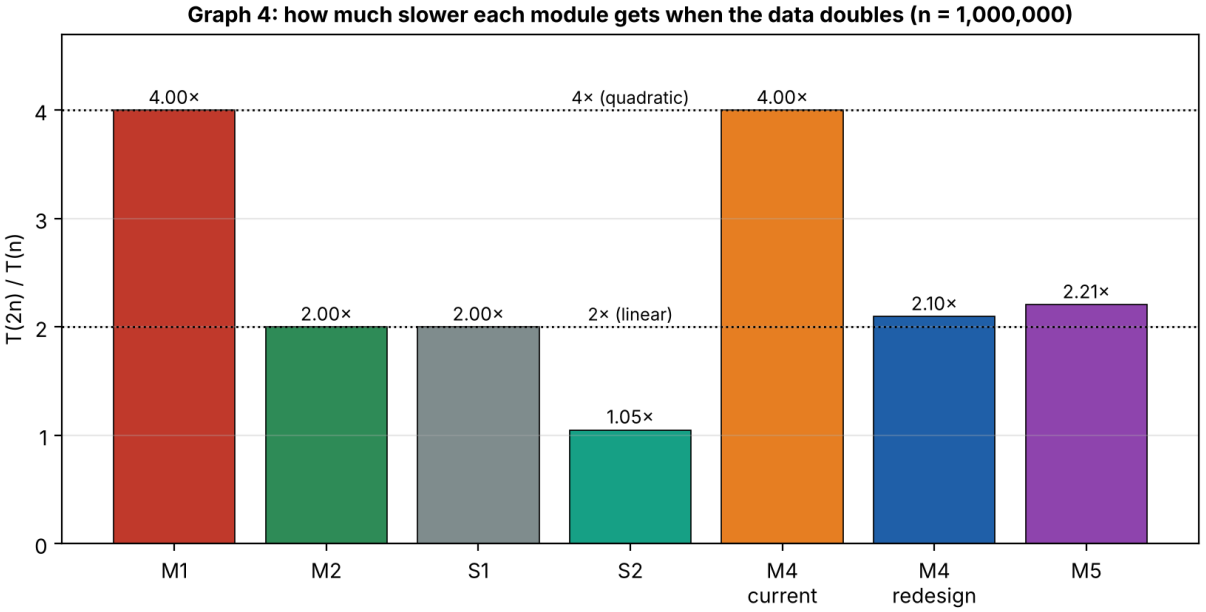



**Reading Graph 2:** at n = 100,000, M1 takes 50 s and M4 (current) 200 s, which is still inside the window. At n = 1,000,000 they take 5,000 s (about 83 minutes) and 20,000 s (about 5.6 hours): data grew 10× but time grew 100×. The M4 redesign needs only about 0.2 s, and M5 about 4 s.

# GRAPHICAL ANALYSIS OF THE SHOPSPHERE MODULES (continued)




**Reading Graph 3:** the actual number of Audit operations in M2 (counted by running the loop for n up to 16,384 and equal to n − 1 for every power of two) follows a straight line parallel to n, while the claimed n log n curve moves away from it as n grows. At n = 1,024 the loop performs 1,023 Audits but the claim predicts 10,240, so the gap is a factor of log n. This confirms that the tight bound is Θ(n).




**Reading Graph 4:** when the data doubles at n = 1,000,000, M1 and the current M4 become 4 times slower, which is the signature of Θ(n²). The M4 redesign (2.10×) and M5 (2.21×) become only a little more than twice as slow, M2 and S1 exactly twice, and the binary search S2 only 1.05×, i.e. one extra comparison.

**Proof that f(n) is not O(n):**

- Assume f(n) = O(n). Then there are constants c and n with f(n) ≤ c·n for all n ≥ n .
- Since f(n) ≥ 3n², this gives 3n² ≤ c·n, which means n ≤ c/3.
- But we can choose n larger than both n and c/3 (for example n = max(n , c/3) + 1), and this breaks the inequality.
- This is a contradiction, so **f(n) is not O(n)**. ∎

## (g) Scalability Decision (n = 1,000,000)

Time window = 10 minutes = 600 s.
Operations allowed = 600 × 10 = **$6 \times 10^{1}$** .

| Module | Operations | Time at 10 ops/s | Fits in 10 min? |
|---|---|---|---|
| M1: n(n−1)/2 | ≈ $5 \times 10^{11}$ | ≈ 5,000 s ≈ 83 min | **No** |
| M2: $2^{2}$ − 1 | ≈ 1.05 × 10 | ≈ 0.01 s | Yes |
| S2: log n + 1 | ≈ 20 | ≈ 2 × 10 s | Yes |
| M4 current: 2n² − n (exact, with T(1) = 1) | ≈ $2 \times 10^{12}$ | ≈ 20,000 s ≈ 5.6 hours | **No** |
| M4 redesign: n log n + n | ≈ 2.1 × 10 | ≈ 0.2 s | Yes |
| M5: about n log² n | ≈ 4 × 10 | ≈ 4 s | Yes |

**Modules that fail the 10-minute window: M1 and M4 (current design).**

**Recommendations**

1. **M1 must be redesigned.**
   - Use a hash table with the key (title + seller). Take each product, check whether its key is already in the table, and if yes mark it as a duplicate, otherwise insert it.
   - Time becomes O(n) on average (about 10 operations, under a second), at the cost of O(n) extra space.
   - Alternative: sort the products by (title, seller) in O(n log n), then compare only neighbouring products.
   - Result: from about 83 minutes down to under a second.
2. **M4 must adopt the proposed redesign.** Removing replication changes Θ(n²) to Θ(n log n), from about 5.6 hours to about 0.2 seconds.
3. **M3 should use S2 (the sorted index).** About 20 comparisons instead of an average of hundreds of thousands.
4. **M2 and M5 can stay as they are.** M2 is already linear. M5 finishes in seconds. If wanted later, M5 can be improved to Θ(n log n) by sorting once instead of at every level.

**Conclusion:** modules with Θ(n²) growth (M1 and the current M4) will not scale to 1,000,000 products. Modules with Θ(log n), Θ(n) or Θ(n log n) growth are safe.